Übungsblatt 8: I/O & Systemarchitektur

Mikrocomputertechnik 1

Zielsetzung

In diesem Übungsblatt brechen Sie die Isolation der CPU auf. Im ersten Teil (Ripes) steuern Sie über Memory-Mapped I/O (MMIO) direkt Hardware-Register wie Schalter und LEDs an. Im zweiten Teil (Venus) nutzen Sie System Calls, um das Betriebssystem als Pförtner für Konsolenausgaben (Drucken von Text und Zahlen) um Hilfe zu bitten.

Aufgabe 1: Hardware-Kommunikation und Traps

Beantworten Sie die folgenden Fragen zu den Systemgrundlagen der Vorlesung.

MMIO und das `volatile`-Problem

In der C-Programmierung muss bei Pointern auf Memory-Mapped-I/O-(MMIO)-Adressen zwingend das Schlüsselwort `volatile` genutzt werden. Erklären Sie, warum I/O-Register „lebendig“ sind und was passiert, wenn der Compiler eine Leseschleife ohne `volatile` optimiert.

Polling vs. Interrupts

Was ist der größte Nachteil von Polling (Busy Waiting) auf Systemebene? Warum vergleicht man Interrupts stattdessen mit einer Türklingel?

Das CSR-System

Bei einem Interrupt springt die CPU in den Trap-Handler. Welche Aufgabe übernehmen dabei die speziellen Schattenregister `mtvec` und `mepc`?

Ihre Lösung

(Notieren Sie Ihre Antworten hier.)

Aufgabe 2: Bare-Metal I/O in Ripes (Switches & LEDs)

Öffnen Sie Ripes (Single-Cycle Processor). Wechseln Sie in den Reiter **I/O** und fügen Sie über das Plus-Symbol **Switches** und eine **LED Matrix** hinzu. Lesen Sie die Basisadressen unten links ab (typischerweise Switches: `0xF0000000`, LEDs: `0xF0000020`).

Aufgabenstellung

Programmieren Sie eine Endlosschleife, die permanent die Switches ausliest.

1. Filtern Sie mit einer logischen Maske (`andi`) exakt Schalter Nummer 3 (Bit 3) heraus.
2. Wenn der Schalter gedrückt ist, schalten Sie die erste LED in der Matrix auf Rot (Farbwert `0x00FF0000`).
3. Wenn der Schalter nicht gedrückt ist, schalten Sie die LED ab (`0x00000000`).

Ihre Lösung

(Fügen Sie Ihren Assembler-Code hier ein.)

Aufgabe 3: OS-Syscalls in Venus (Konsole & EVA)

Schließen Sie Ripes und öffnen Sie den Venus-Simulator. Wir verlassen Bare-Metal und rufen das Betriebssystem (`ecall`) an, um Text auf die Konsole zu drucken.

Achtung: Die Venus-ABI erwartet die Service-ID in `a0` und das Argument in `a1`.

Aufgabenstellung

1. Legen Sie im `.data`-Segment einen nullterminierten String an: `.string "Ergebnis: "` (oder `.asciiz`).
2. Rufen Sie Service 4 (Print String) auf. `a1` benötigt die Speicheradresse des Strings (`la`).
3. Laden Sie die Konstante 21 in ein temporäres Register und verdoppeln Sie sie per Schiebeoperation (`slli`) oder Addition.
4. Rufen Sie Service 1 (Print Integer) auf, um die berechnete Zahl auszudrucken.
5. Beenden Sie das Programm sauber mit Service 10 (Exit).

Ihre Lösung

(Fügen Sie Ihren Assembler-Code hier ein.)

Aufgabe 4: Die Clobbering-Gefahr bei `ecall`

Ein Kommilitone hat folgenden Venus-Code geschrieben, um dreimal das Wort „Hallo“ zu drucken. Das Programm druckt „Hallo“ jedoch endlos (oder verhält sich unzuverlässig) und beendet sich nicht sauber.

```
.data
text: .string "Hallo\n"

.text
    li t0, 3            # Schleifenzähler
Loop:
    beq t0, zero, End

    li a0, 4            # Print String
```

```
    la a1, text
    ecall

    addi t0, t0, -1     # Zähler dekrementieren
    j Loop

End:
    li a0, 10
    ecall
```

Aufgabenstellung

1. Finden Sie den Fehler: Warum ist der Schleifenzähler in `t0` laut ABI nach dem `ecall` nicht garantiert — auch wenn Venus zufällig funktioniert?
2. Wie können Sie den Fehler am einfachsten beheben, ohne den Stack zu nutzen?

Ihre Lösung

(Notieren Sie Ihre Analyse hier.)